Data Structures Assignments

1. Write a program to display the factorial value of a given number taken as input from the user (non-recursive implementation).

```
Algorithm main ()
        {
            // assuming non-negative integers as input
            fact = 1;
            display ("Enter a value");
            input(n);
            for (i = 1; i <= n; i++)
              fact = fact*i;
            display ("The factorial of the given number is");
            display(fact);
         }
```

2. Write a program to display the first 'n' terms Fibonacci series (where the value of 'n' taken from the user as an input) [non-recursive implementation]

```
Algorithm fib(n)
        {
            // initialize first and second terms
            t1 = 0; t2 = 1;
            nextTerm = t1 + t2;      // initialize the nextTerm
            display (t1, t2);          // display first two terms
            for (i = 3; i <= n; i++)
            {
               display (nextTerm);
                t1 = t2;
                t2 = nextTerm;
                nextTerm = t1 + t2;
            } // display all the terms from 3rd to nth
        }
Algorithm main ()
        {
            display ("Enter the number of terms");
            input(n);
            fib(n);
        }
```

3. a) Calculate the factorial of a non-negative numbers (taken from the user as input). A recursive implementation but not tail recursive implementation.

```
Algorithm factorial(n)
        {
            If (n == 0)
              return 1;
            return n*factorial(n-1);
        }
```

b) Calculate the factorial of non-negative number (taken from the user as input). A tail recursion implementation.

```
Algorithm factorial1(n, accumulator)
        {
            If (n == 0)
              return accumulator;
            return factorial1(n-1, n*accumulator);
        }
Algorithm factorial(n)
        {
            return factorial1(n, 1);
        }
```

4. Write a recursive function (along with its algorithm) which displays the first 'n' terms of the Fibonacci series (starting from 0) and return the value of the n^{th} terms of Fibonacci series. Also specify whether your algorithm/function is tail recursive.

```
Algorithm fibonacci (n, a, b)
        {
            If (n == 1)
            {
              display(a);
               return a;
            }
            else
            {
              display (a);
              return fibonacci (n-1, b, a + b);
            }
        }
```

5. Write function(s) (along with its algorithm) which displays a suitable menu to the user for performing the following tasks on a list of elements stored in an array, depending upon the user's choice:

   a) Store a list of 'n' real numbers(elements) provided by the user as input in the array (where the values of 'n' is also provided by the user as input).

b) Search the number (provided by the user as input) and display its location in the list.
c) Insert an element in the list at the location specified by the user as input.

d) Delete an element from the list where the location of the deleted element in the list is specified by the user as input.

e) Display the content of the list.

6. a) Write function(s) (along with algorithm(s)) which displays a suitable menu to user for performing the following tasks on the list of names (stored in array) depending upon user's choice. The name may consist of first name followed by middle name and surname, separated by a blank space:

   i. Store a list of names (elements) [provided by the user as input] in the array (where the values of 'n' is also provided by the user as input)

   ii. Search a given name (provided by the user as input) and display its location in the list.

   iii. Insert a given name in the list at the location specified by the user as input.

   iv. Delete a name from the list where the location of deleted element in the list is specified by the user as input.

   v. Display the content of the list.

   b) Write function(s) (along with its algorithm(s)) to perform the addition of two polynomials (taken from the user as input) [where each polynomial is stored in the array of elements in the form (m, e_{m-1}, b_{m-1}, e_{m-2}, b_{m-2}, ... , e_0, b_0) where, the first entry ***m*** represents the number of non-zero terms (or the number of non-zero coefficients) followed by two entries (e_i, b_i) for each non-zero term (representing an exponent-coefficient pair) in the order $e_{m-1} > e_{m-2} > \ldots > e_0 > = 0$] store the resultant polynomial and display the resultant polynomial.

7. a) Write function(s) [along with its algorithm(s)) which perform multiplication of 2 matrices (taken as input from the user) and display the resultant matrix.

   b) Let a sparse matrix be stored in an array X (0:t, 0:2) where t is the number of non-zero elements. The elements X (0, 0) and X (0,1) contain the total number of columns of the matrix respectively. X (0, 2) contains the total number if non-zero elements of the matrix. Let X (i, 0), X (i, 1) & X (i, 2) contain row number, column number & the value of the non-zero element in the matrix respectively, for 1 <= i <= t. Write function(s) (along with its algorithm(s)) to store two sparse matrices (taken as input from the user) in the form (as specified above using X), perform addition of these matrices, store the result in the form (as specified above using X) and display the result (in the form which is usually understood) by an user unfamiliar with the representation specified using X.